

Simulating and Optimising Emergency-Department Patient Flow

Replicating Lv et al. (2026) — and going four steps further

Sudip Kumar Saha Ahnaf Akib Tanim Md. Shahjalal Rumman Md. Dulal
Hossain Md. Shoaib Hossain

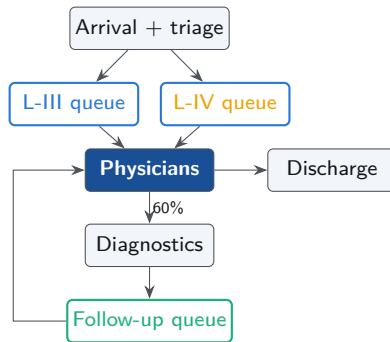
CSE 402 — Numerical Analysis, Simulation and Modeling Sessional
Group 02, Section C

Base paper: Lv, Liu, Yan & Wang (2026), *Healthcare* 14(1):99

Only triage **Level III** (urgent) and **Level IV** (non-urgent) patients are modelled — Levels I–II are critical and bypass the queue entirely.

Arrivals	non-homogeneous Poisson, weekly cycle — 1.7 to 23 patients/h
Triage mix	25% Level III / 75% Level IV
Waiting targets	$T_3 = 30$ min $T_4 = 120$ min
Initial consult	5–15 min (mean 9.1)
Diagnostics	60% of patients: lab, X-ray, CT, ultrasound (+ report delay)
Follow-up consult	5–25 min (mean 12.8)
Physicians	5 morning / 5 afternoon / 3 night
One run	1 week $\approx$ 2,280 patients

These imply a system at **87% load** — where every extra point of utilisation costs minutes of waiting.



A physician becomes free. **Which of the three queues do they take from?** The three policies differ only in how they answer that.

Policy	Rule	Consequence
IFP Initial-First	New arrivals always first (L-III, then L-IV, then follow-ups)	Nobody breaches target, but follow-ups starve
ALT Alternating 1:1	At most half the doctors on new arrivals at once	L-III waits collapse; L-IV breaches
SBP Slack-Based	Urgent once waited $T_\ell - k_\ell$; two tunable dials (k_1, k_2)	Best of the three — <i>if</i> tuned well

The slack idea, in one sentence

k_ℓ is a *head start*: with $T_3 = 30$ and $k_1 = 13.1$ a Level III patient becomes “urgent” after $30 - 13.1 = 16.9$ min — the system starts rescuing them **13.1 minutes before** they would breach. Bigger k = earlier rescue = more protection for that class, at everyone else’s expense.

Reported performance

	Wait	L-III	SL ₄	Util.
IFP	46.26	44.14	100%	85.43%
ALT	37.88	6.82	89.3%	85.41%
SBP	35.25	10.48	100%	84.98%

SBP wins: 46.26 → 35.25 min, a **−23.8%** cut, with zero service-level violations.

How it tuned (k_1, k_2)

Stage	Step	Best
Coarse lattice	1.0	(13, 2) — 36.38 min
Fine lattice	0.1	(13.1, 2.1) — 35.25 min

Validated with paired t -tests, Bonferroni correction and Cohen's d (all > 1.5).

Its other conclusions

- Equipment runs at **14–19%**, physicians at **85%** ⇒ **doctors are the bottleneck**
- More staff shortens waits, with diminishing returns
- The ranking survives a different service-time distribution

What it does *not* do

- Optimise the **staffing level** — it only probes it
- Compare against a **provably optimal** rule
- Use a real **numerical optimiser**
- Apply any **variance reduction**
- Release its engine or its dataset

Those five gaps are what we take on next.

What we did

Rebuild it from scratch, then push it four steps further

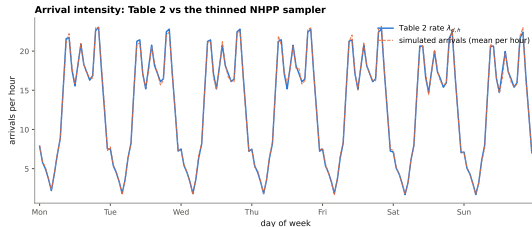
What	Why it matters
1 Rebuild & validate the whole simulator from the published parameters alone	The paper releases neither code nor data — we check every KPI it reports
2 Add a provably optimal scheduling rule (the $c\mu$ rule)	The paper's three policies are all hand-designed heuristics
3 Replace brute-force search with a Nelder–Mead optimiser under a real constraint	The lattice costs 50,415 simulations; we ask how few we need
4 Put a price on waiting and salary, then optimise the staffing roster	“More doctors is better” has no stopping point
5 Common random numbers — give every policy the identical simulated week	Sharpens every comparison without a single extra run

~7,900 lines of Python; hand-rolled `heapq` event loop (no SimPy); 33 automated correctness checks, all passing.

1. Does the rebuild match the paper? Structure yes, absolute level no

	Ours	Paper	Dev.
Reproduced closely			
ALT L-III wait	6.05	6.82	-11%
Utilisation	86.3%	85.4%	+1%
Lab equipment	14.73%	14.36%	+0.4pp
CT equipment	18.18%	17.64%	+0.5pp
IFP service level	100/100	100/100	exact
Runs high			
IFP mean wait	52.43	46.26	+13%
SBP mean wait	44.08	35.25	+25%

Why? The published parameters imply load $\rho \approx 0.87$. Near saturation, waiting time is hypersensitive to the last point of utilisation — and the source dataset is not public. **All 6 structural findings reproduce.**



The arrival process, validated over 200 simulated weeks: relative error **1.67%**, correlation **0.9991**.

2. A rule with a proof behind it The $c\mu$ rule (Cox & Smith, 1961)

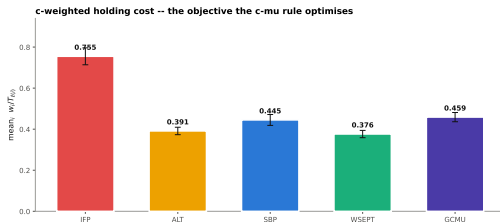
If class ℓ costs c_ℓ per minute of delay and takes $1/\mu_\ell$ to serve, the *cost-minimising* policy always serves the class with the largest

$$I_\ell = c_\ell \cdot \mu_\ell$$

We set $c_\ell = 1/T_\ell$ — a minute hurts in inverse proportion to what that class tolerates.

Class	T_ℓ	μ_ℓ	$c_\ell \mu_\ell$
L-III initial	30	0.110	0.003666
Follow-up	60	0.078	0.001298
L-IV initial	120	0.110	0.000916

Indices are constants $\Rightarrow$ fixed priority. We also implement the *generalised* $c\mu$ rule, $I_\ell = 2w\mu_\ell/T_\ell^2$, which adapts to how long the front patient has waited — and has **zero parameters to tune**.



Judged on *its own* objective — waiting measured in units of each patient's tolerance — the $c\mu$ rule wins: **0.376** vs IFP's **0.755**, a 50% reduction. On this system it also gives the lowest raw mean wait (42.32 min).

3. A real optimiser instead of brute force

Hand-implemented simplex + penalty function

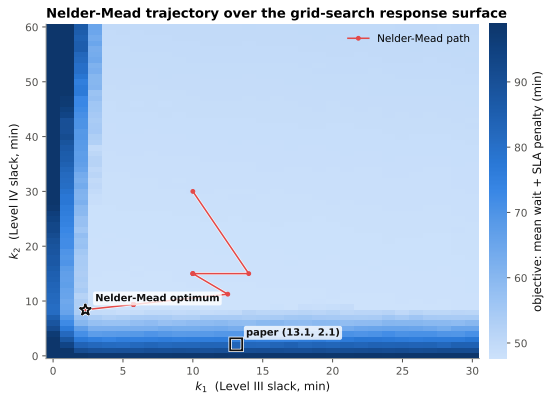
Minimise, subject to $SL_\ell \geq 95\%$:

$$F = \overline{W} + \mu \sum_\ell \max(0, SL_\ell^{\min} - SL_\ell)$$

- Same objective, same weeks as the grid search
- Budget counted in simulations, not seconds
- Noise handled by sample-average approximation

Method	Points	Sims
Grid search (paper's)	2,332	50,415
Nelder–Mead	53	2,650

19× fewer simulations; the two optima differ by 0.006 min. Matches scipy to 0.0000.



The simplex walks down to the feasible optimum. The dark band is the service-level penalty — and **the paper's own published (13.1, 2.1) sits inside it**: it reaches only 89.4% on Level IV, because $k_2 = 2.1$ rescues a patient at minute 117.9 of 120.

4. How many doctors should the ED roster? Put a price on the trade-off

$$C(R, k_1, k_2) = \underbrace{\alpha H_{\text{phys}}}_{\text{salary}} + \underbrace{\beta W_{\text{total}}}_{\text{waiting}} + \underbrace{\gamma V_{\text{SLA}}}_{\text{breaches}}$$

$$R = (R_{\text{morning}}, R_{\text{afternoon}}, R_{\text{night}})$$

	Roster	Wait	SL_4	Cost/wk
Paper	(5,5,3)	44.18	95.6%	139,391
S_5	(6,7,5)	4.32	100%	124,082
Ours	(6,6,3)	10.97	99.9%	110,513

One more doctor on each *day* shift, with thresholds re-tuned to (12.2, 23.4): **−20.7% weekly cost.**

Why re-tune inside each roster? The best (k_1, k_2) for a 3-doctor night is not the best for 5 — k_2 moves from 8.4 to 23.4. Holding them fixed would score every alternative roster at someone else's settings.

Does the answer survive the assumptions?

Cost scenario	Winner	vs (5,5,3)
Baseline	(6,6,3)	−20.7%
Waiting $\times 4$	(6,7,3)	−53.2%
Waiting $\times 1/4$	(5,6,3)	−5.2%
Doctors $\times 1.5$	(6,6,3)	−12.4%
Breach $\times 10$	(6,6,3)	−39.5%

The winner *moves* with the weights — **that is itself the finding.** The right staffing level depends on how a hospital prices a waiting minute against a salary. But **every** weighting beats the paper's (5,5,3).

5. Sharper statistics for free Common random numbers

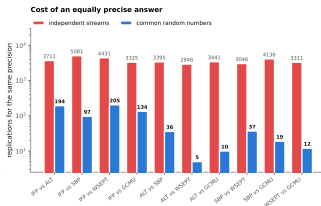
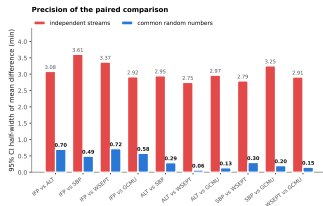
$$\text{Var}(A - B) = \text{Var}(A) + \text{Var}(B) - 2 \text{Cov}(A, B) \quad (\text{Cov} = 0 \text{ with independent streams})$$

Give both policies the **identical week** and a busy week inflates both together: $\text{Cov} \rightarrow 1$, so the variance of the *difference* collapses.

Correlation induced	+0.996
Variance removed	98.95%
Efficiency gain	95.8×

The whole patient stream is generated *before* the run, so patient #1723 is bit-identical under every policy. Not a bias: each policy's own mean and s.d. are unchanged.

What Common Random Numbers buy (target CI half-width 0.5 min)



Confidence intervals shrink 4–40×; replications needed for ± 0.5 min precision fall from thousands to dozens. Practical reading: **20–30 CRN replications** beat the paper's 100 independent ones.

Results: all six policies, one table 100 paired replications

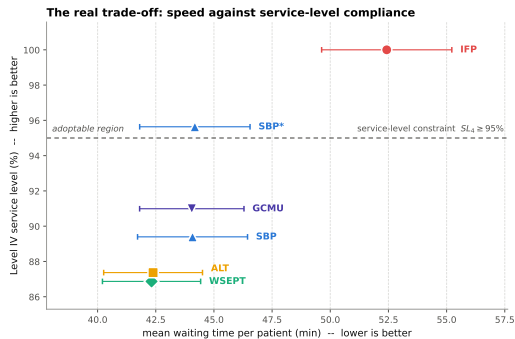
Policy	Origin	Wait	L-III	L-IV	Holding	SL ₄	Meets SLA?
IFP	base paper	52.43	50.88	52.94	0.7550	100.0%	yes
ALT	base paper	42.39	6.05	54.61	0.3913	87.4%	no
SBP	base paper (13.1, 2.1)	44.08	12.35	54.74	0.4450	89.4%	no
SBP*	ours (2.31, 8.37)	44.18	14.14	54.27	0.4570	95.6%	yes
WSEPT	ours, $c\mu$	42.32	3.76	55.30	0.3763	86.9%	no
Gc μ	ours, gen. $c\mu$	44.05	14.59	53.94	0.4588	91.0%	no

There is no single winner — and that is the result

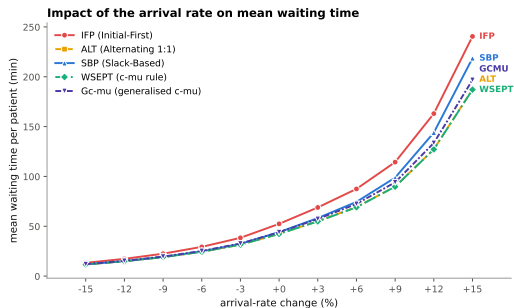
Shortest raw wait	WSEPT (42.32 min)	the $c\mu$ rule
Lowest holding cost	WSEPT (0.3763)	its own optimal objective
Fastest that meets the SLA	SBP* (44.18 min)	the only adoptable one

All 15 pairwise differences tested with paired t -tests, Bonferroni-corrected ($\alpha_{\text{adj}} = 0.00333$); 12 of 15 significant.

Comparative analysis: the whole study in two pictures



Speed vs. compliance. Only policies above the dashed 95% line are adoptable. IFP is safe but slow; the fast rules all breach; **only SBP* is both.**



Robustness. Under light load the policies are interchangeable; as the ED saturates the choice of rule is worth minutes per patient. Reproduces the paper's Figure 4 — as do its staffing and lognormal-service-time checks.

What we established

- The paper's model **reproduces structurally** — all 6 findings, equipment table within 0.5 pp
- A **provably optimal** rule beats all three heuristics on the delay-cost objective
- Nelder–Mead reaches the same optimum with **19× fewer** simulations
- The cost-optimal roster **(6,6,3)** saves **20.7%** per week
- CRN removes **98.9%** of comparison variance

The finding we did not expect

On our model the paper's own published thresholds (13.1, 2.1) **violate the 95% Level IV constraint**.

Published scheduling parameters are properties of the system they were fitted to — not transferable constants.

Next: per-shift thresholds, a re-entrant $c\mu$ bound, and a two-stage stochastic program for the roster.

Code, all result tables, figures and a full provenance log:
github.com/Ahnaaf-Akib-Tanim/Numeric-Project-submission

Thank you

Questions?

Sudip Kumar Saha ▪ Ahnaf Akib Tanim ▪ Md. Shahjalal Rumman ▪ Md. Dulal Hossain ▪ Md.
Shoaib Hossain

CSE 402 — Group 02, Section C